03장 프로그램의 구조를 쌓는다! 제어문

03장에서는 if, while, for 등의 제어문에 대해 배운다. 제어문을 배우기 전에 집 짓는 일을 생각해 보자. 집을 지을 때 나무, 돌, 시멘트 등은 재료가 되고 철근은 뼈대가 된다. 프로그램을 만드는 것도 집 짓기와 비슷하다. 나무, 돌, 시멘트 같은 재료는 자료형에 해당하고, 뼈대를 이루는 철근은 제어문에 해당한다. 자료형을 바탕으로 제어문을 활용해 프로그램의 구조를 만들어 보자.

목차

- 03-1 if문
- 03-2 while문
- 03-3 for문
- 03장 되새김 문제

03-1 if문

먼저 대표적인 조건문인 if 문부터 알아보자.

- if 문은 왜 필요할까?
- if 문의 기본 구조
- 들여쓰기 방법 알아보기
- 조건문이란 무엇인가?
    - 비교 연산자
    - and, or, not
    - in, not in
- 다양한 조건을 판단하는 elif
- match-case 문
- 비교 연산자의 연쇄 사용
- 조건부 표현식

if 문은 왜 필요할까?

다음과 같은 상황을 생각해 보자.

```
'돈이 있으면 택시를 타고 가고, 돈이 없으면 걸어간다.'
```

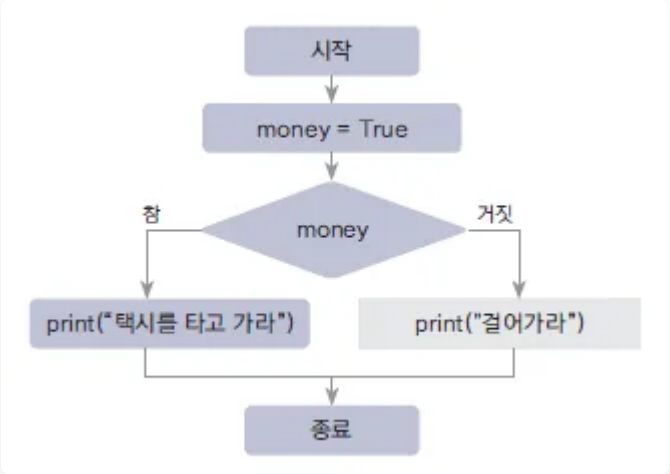
누구에게나 일어날 수 있는 상황이다. 프로그래밍도 마찬가지다. 주어진 조건을 판단한 후 그 상황에 맞게 처리해야 할 경우가 생긴다. 이렇듯 프로그래밍에서 조건을 판단하여 해당 조건에 맞는 상황을 수행하는 데 쓰는 것이 바로 if 문이다.

파이썬에서는 위 상황을 다음과 같이 표현할 수 있다.

```
>>> money = True
>>> if money:
...     print("택시를 타고 가라")
... else:
...     print("걸어 가라")
...
택시를 타고 가라
```

money에 True를 대입했으므로 money는 참이다. 따라서 if 문 다음 문장이 수행되어 '택시를 타고 가라'가 출력된다.

다음 순서도는 '택시를 타고 가라'라는 문장이 출력되는 과정을 보여 준다.



if 문의 기본 구조

다음은 if와 else를 사용한 조건문의 기본 구조이다.

```
if 조건문:
    수행할_문장1
    수행할_문장2
    ...
else:
    수행할_문장A
    수행할_문장B
    ...
```

조건문이 참이면 if 문 바로 다음 문장(if 블록)을 수행하고, 거짓이면 else 문 다음 문장(else 블록)을 수행한다. else 문은 if 문 없이 독립적으로 사용할 수 없다.

들여쓰기 방법 알아보기

if 문을 만들 때는 `if 조건문:` 바로 다음 문장부터 if 문에 속하는 모든 문장에 들여쓰기(indentation)를 해야 한다. 다음 예를 보면 조건문이 참일 경우 '수행할_문장1', '수행할_문장2', '수행할_문장3' 모두 들여쓰기했다. 다른 프로그래밍 언어를 사용해 온 사람들은 파이썬에서 들여쓰기를 무시하는 경우가 많으므로 더 주의해야 한다.

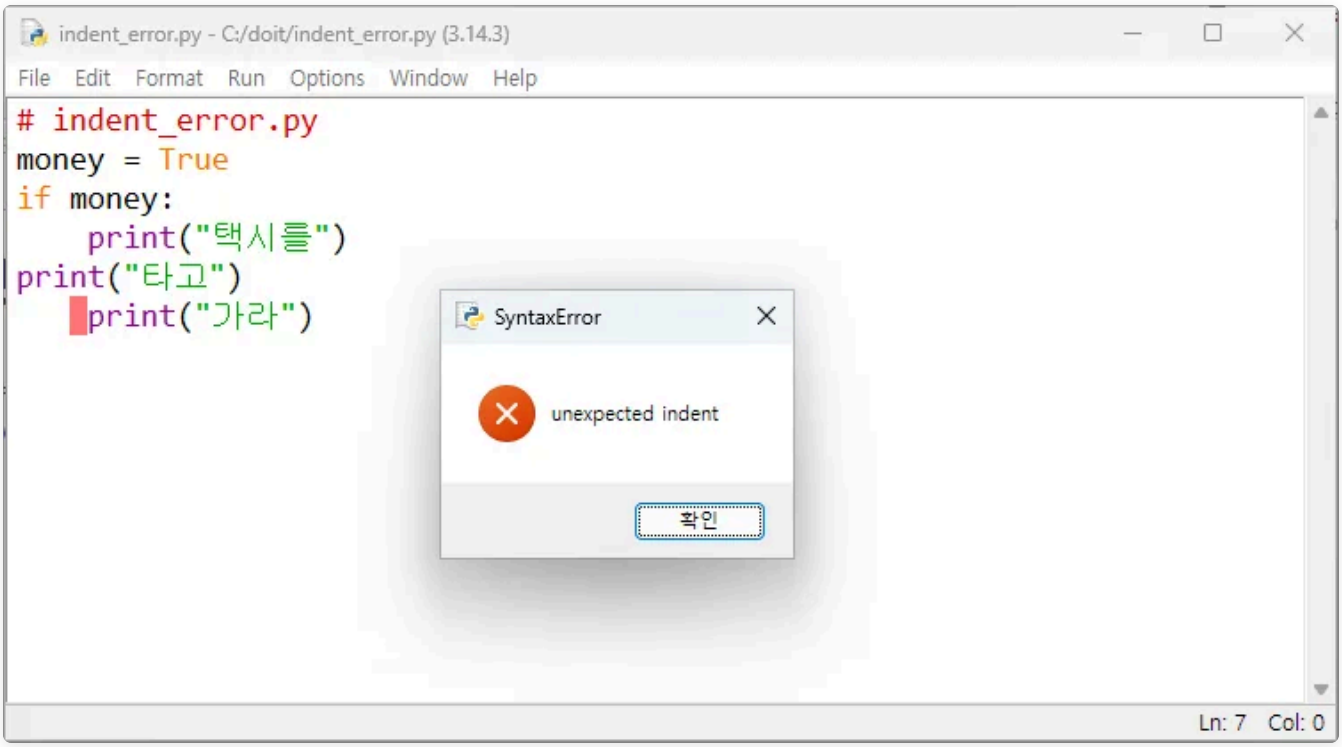
```
if 조건문:
    수행할_문장1
    수행할_문장2
    수행할_문장3
```

다음처럼 작성하면 오류가 발생한다. '수행할_문장2'를 들여쓰기하지 않았기 때문이다.

```
if 조건문:
    수행할_문장1
수행할_문장2
    수행할_문장3
```

IDLE 에디터에서 다음과 같이 작성하여 실행해 보자. 오류가 발생하는지 확인만 하면 되므로 IDLE 에디터 창에 코드를 작성하고 `[F5]`를 눌러 바로 실행하자.

```
# indent_error.py
money = True
if money:
    print("택시를")
print("타고")
    print("가라")
```



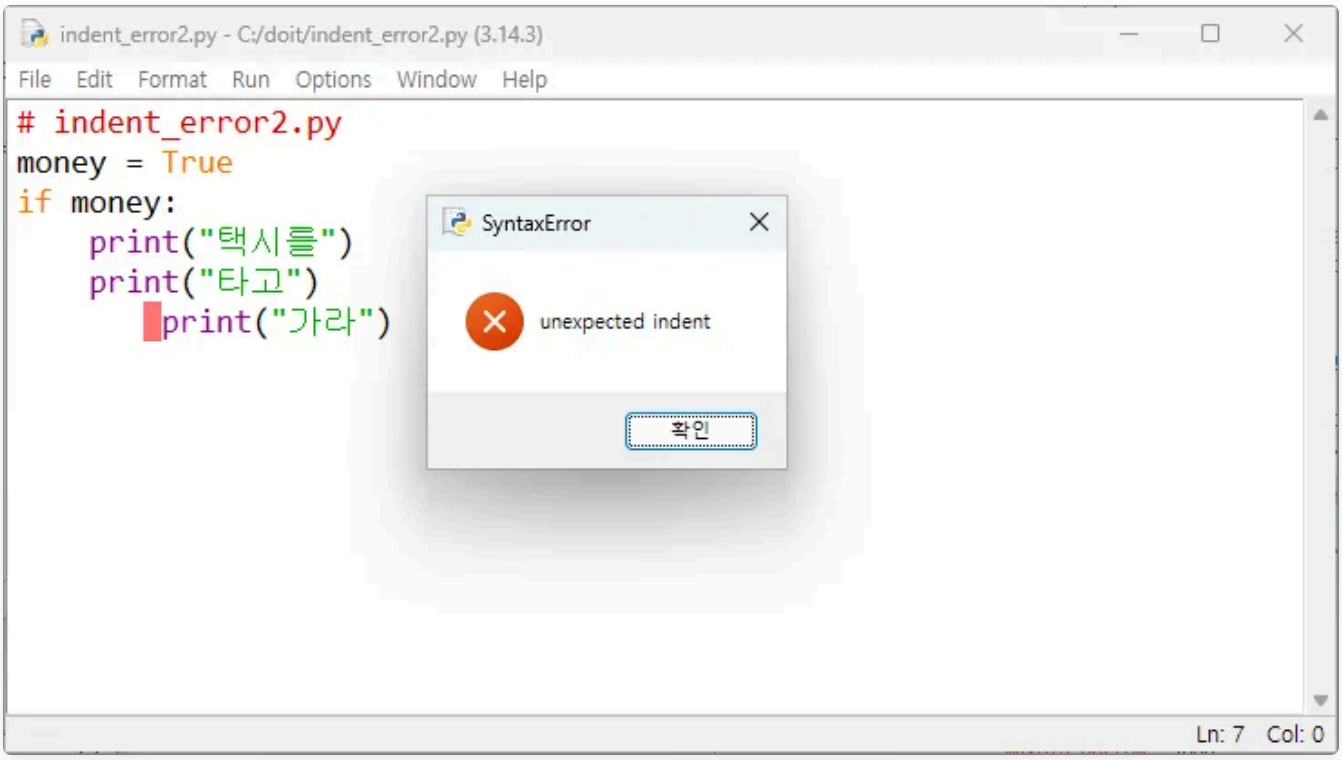
> *들여쓰기 오류가 발생하는 것을 확인할 수 있다.*

다음 경우에도 오류가 발생한다. '수행할_문장3'을 들여쓰기했지만, '수행할_문장1'이나 '수행할_문장2'와 들여쓰기 깊이가 다르다. 즉, 들여쓰기는 언제나 같은 깊이로 해야 한다.

```
if 조건문:
    수행할_문장1
    수행할_문장2
        수행할_문장3
```

IDLE 에디터에서 다음과 같이 작성하여 실행해 보자.

```
# indent_error2.py
money = True
if money:
    print("택시를")
    print("타고")
        print("가라")
```



마찬가지로 들여쓰기 오류가 발생한다.

그렇다면 들여쓰기는 공백 문자([Spacebar])로 하는 것이 좋을까, 탭 문자([Tab])로 하는 것이 좋을까? 이에 대한 논란은 아직도 계속되고 있다. 공백 문자 쪽과 탭 문자 쪽 모두가 동의하는 내용은 2가지를 혼용해서 쓰지 말자는 것이다. 공백 문자로 할 것이라면 공백으로 통일하고, 탭 문자로 할 것이라면 탭으로 통일하자. 탭이나 공백은 눈으로 보이지 않기 때문에 혼용해서 쓰면 오류의 원인이 되므로 주의하자.

> *요즘 파이썬 커뮤니티에서는 들여쓰기를 할 때 공백 문자 4개를 사용하는 것을 권장한다. 또한 대부분의 파이썬 에디터는 탭 문자로 들여쓰기를 하더라도 탭 문자를 공백 문자 4개로 자동 변환하는 기능을 갖추고 있다.*

점프 투 파이썬

조건문 다음에 콜론(:)을 잊지 말자!

if 조건문 뒤에는 반드시 콜론(:)이 붙는다. 어떤 특별한 의미가 있다기보다는 파이썬의 문법 구조이다. 앞으로 배울 while이나 for, def, class도 문장의 끝에 콜론(:)이 항상 들어간다. 초보자들은 이 콜론(:)을 빠뜨리는 경우가 많으므로 특히 주의하자.

파이썬이 다른 언어보다 보기 쉽고 소스 코드가 간결한 이유는 콜론(:)을 사용하여 들여쓰기를 하도록 만들었기 때문이다. 하지만 이는 숙련된 프로그래머들이 파이썬을 처음 접할 때 가장 혼란스러워하는 부분이기도 하다. 다른 언어에서는 if 문에 속한 문장들을 {}로 감싸지만, 파이썬에서는 들여쓰기로 해결한다는 점을 기억하자.

조건문이란 무엇인가?

if 조건문에서 '조건문'이란 참과 거짓을 판단하는 문장을 말한다.

앞에서 살펴본 택시 예제에서 조건문은 money가 된다.

```
>>> money = True
>>> if money:
```

money는 True이기 때문에 조건이 참이 되어 if 문 다음 문장을 수행한다.

비교 연산자

이번에는 조건문에 비교 연산자(<, >, ==, !=, >=, <=)를 쓰는 방법에 대해 알아보자.

다음 표는 비교 연산자를 정리한 것이다.

비교연산자	설명
x < y	x가 y보다 작다.
x > y	x가 y보다 크다.
x == y	x와 y가 같다.
x != y	x와 y가 같지 않다.
x >= y	x가 y보다 크거나 같다.
x <= y	x가 y보다 작거나 같다.

이제 이 연산자를 어떻게 사용하는지 알아보자.

```
>>> x = 3
>>> y = 2
>>> x > y
True
>>>
```

x에 3, y에 2를 대입한 후 `x > y`라는 조건문을 수행하면 True를 반환한다. `x > y` 조건문이 참이기 때문이다.

```
>>> x < y
False
```

위 조건문은 거짓이기 때문에 False를 반환한다.

```
>>> x == y
False
```

x와 y는 같지 않다. 따라서 위 조건문은 거짓이므로 False를 반환한다.

```
>>> x != y
True
```

x와 y는 같지 않다. 따라서 위 조건문은 참이므로 True를 반환한다.

사용법을 알아봤으므로 이제 응용해 보자. 앞에서 살펴본 택시 예제를 다음처럼 바꾸려면 어떻게 해야 할까?

```
만약 3000원 이상의 돈을 가지고 있으면 택시를 타고 가고, 그렇지 않으면 걸어가라.
```

이 상황은 다음처럼 프로그래밍할 수 있다.

```
>>> money = 2000
>>> if money ⩾ 3000:
...     print("택시를 타고 가라")
... else:
...     print("걸어가라")
...
걸어가라
>>>
```

`money ⩾ 3000` 조건문이 거짓이 되기 때문에 else 문 다음 문장을 수행하게 된다.

and, or, not

조건을 판단하기 위해 사용하는 다른 연산자로는 and, or, not이 있다. 이러한 논리 연산자는 02-7 불 자료형에서 자세히 알아보았다. 각각의 연산자는 다음처럼 동작한다.

연산자	설명
x or y	x와 y 둘 중 하나만 참이어도 참이다.
x and y	x와 y 모두 참이어야 참이다.
not x	x가 거짓이면 참이다.

다음 예를 통해 or 연산자의 사용법을 알아보자.

```
돈이 3000원 이상 있거나 카드가 있다면 택시를 타고 가고, 그렇지 않으면 걸어가라.
```

```
>>> money = 2000
>>> card = True
>>> if money ⩾ 3000 or card:
...     print("택시를 타고 가라")
... else:
...     print("걸어가라")
...
택시를 타고 가라
>>>
```

money는 2000이지만, card가 True이기 때문에 `money ⩾ 3000 or card` 조건문이 참이 된다. 따라서 if 문에 속한 '택시를 타고 가라' 문장이 출력된다.

in, not in

파이썬은 다른 프로그래밍 언어에서 쉽게 볼 수 없는 재미있는 조건문도 제공한다.

in	not in
x in 리스트	x not in 리스트
x in 튜플	x not in 튜플
x in 문자열	x not in 문자열

영어 단어 in의 뜻이 '~안에'라는 것을 생각해 보면 다음 예가 쉽게 이해될 것이다.

```
>>> 1 in [1, 2, 3]
True
>>> 1 not in [1, 2, 3]
False
```

첫 번째 예는 '[1, 2, 3]이라는 리스트 안에 1이 있는가?'라는 조건문이다. 1은 [1, 2, 3] 안에 있으므로 참이 되어 True를 반환한다. 두 번째 예는 '[1, 2, 3] 리스트 안에 1이 없는가?'라는 조건문이다. 1은 [1, 2, 3] 안에 있으므로 거짓이 되어 False를 반환한다.

다음은 튜플과 문자열에 in과 not in을 적용한 예이다.

```
>>> 'a' in ('a', 'b', 'c')
True
>>> 'j' not in 'python'
True
```

이번에는 우리가 계속 사용해 온 택시 예제에 in을 적용해 보자.

```
만약 주머니에 돈이 있으면 택시를 타고 가고, 없으면 걸어가라.
```

```
>>> pocket = ['paper', 'cellphone', 'money']
>>> if 'money' in pocket:
...     print("택시를 타고 가라")
... else:
...     print("걸어가라")
...
택시를 타고 가라
>>>
```

`['paper', 'cellphone', 'money']` 리스트 안에 'money'가 있으므로 `'money' in pocket` 은 참이 된다. 따라서 if 문에 속한 문장이 수행된다.

점프 투 파이썬

조건문에서 아무 일도 하지 않게 설정하고 싶다면?

가끔 조건문의 참, 거짓에 따라 실행할 행동을 정의할 때 아무런 일도 하지 않도록 설정하고 싶을 때가 있다. 다음 예를 살펴보자.

```
주머니에 돈이 있으면 가만히 있고, 주머니에 돈이 없으면 카드를 꺼내라.
```

이럴 때 사용하는 것이 바로 pass이다. 위 예를 pass를 적용해서 구현해 보자.

```
>>> pocket = ['paper', 'money', 'cellphone']
>>> if 'money' in pocket:
...     pass
... else:
...     print("카드를 꺼내라")
...
```

pocket 리스트 안에 money 문자열이 있기 때문에 if 문 다음 문장인 pass가 수행되고 아무런 결괏값도 보여 주지 않는다.

다양한 조건을 판단하는 elif

if와 else만으로는 다양한 조건을 판단하기 어렵다. 다음 예를 보자.

```
주머니에 돈이 있으면 택시를 타고 가고, 주머니에 돈은 없지만 카드가 있으면 택시를 타고 가고, 돈도 없고 카드도 없으면 걸어가라.
```

위 문장을 보면 조건을 판단하는 부분이 두 군데 있다. 먼저 주머니에 돈이 있는지를 판단해야 하고, 주머니에 돈이 없으면 다시 카드가 있는지 판단해야 한다.

if와 else만으로 위 문장을 표현하면 다음과 같다.

```
>>> pocket = ['paper', 'cellphone']
>>> card = True
>>> if 'money' in pocket:
...     print("택시를 타고가라")
... else:
...     if card:
...         print("택시를 타고가라")
...     else:
...         print("걸어가라")
...
택시를 타고가라
>>>
```

언뜻 보기에도 이해하기 어렵고 산만한 느낌이 든다. 이런 복잡함을 해결하기 위해 파이썬에서는 다중 조건 판단을 가능하게 하는 elif를 사용한다.

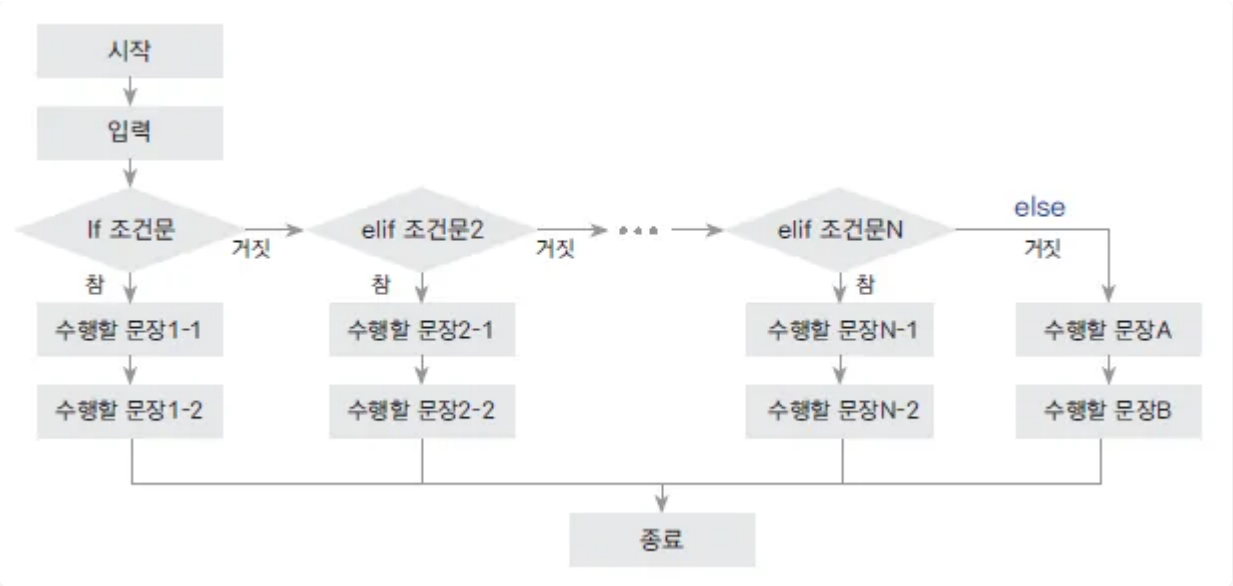
위 예에 elif를 사용하면 다음과 같이 바꿀 수 있다.

```
>>> pocket = ['paper', 'cellphone']
>>> card = True
>>> if 'money' in pocket:
...       print("택시를 타고가라")
... elif card:
...       print("택시를 타고가라")
... else:
...       print("걸어가라")
...
택시를 타고가라
```

elif는 이전 조건문이 거짓일 때 수행된다. if, elif, else를 모두 사용할 때 기본 구조는 다음과 같다.

```
if 조건문:
    수행할_문장1
    수행할_문장2
    ...
elif 조건문:
    수행할_문장1
    수행할_문장2
    ...
elif 조건문:
    수행할_문장1
    수행할_문장2
    ...
...
else:
   수행할_문장1
   수행할_문장2
   ...
```

elif는 개수에 제한 없이 사용할 수 있다.



점프 투 파이썬

if 문을 한 줄로 작성하기

앞의 pass를 사용한 예를 보면 if 문 다음에 수행할 문장이 한 줄이고 else 문 다음에 수행할 문장도 한 줄밖에 되지 않는다.

```
>>> if 'money' in pocket:
...     pass
... else:
...     print("카드를 꺼내라")
...
```

이렇게 수행할 문장이 한 줄일 때 좀 더 간략하게 코드를 작성하는 방법이 있다.

```
>>> pocket = ['paper', 'money', 'cellphone']
>>> if 'money' in pocket: pass
... else: print("카드를 꺼내라")
...
```

if 문 다음에 수행할 문장을 콜론(:) 뒤에 바로 적었다. else 문 역시 마찬가지이다.

match-case 문

> *match-case 문은 파이썬 3.10 버전부터 사용할 수 있다.*

if-elif-else 문으로 여러 조건을 판단할 수 있지만, 하나의 변수 값에 따라 분기하는 경우에는 코드가 다소 길어질 수 있다. 다음 예를 보자.

```
학점이 A이면 "탁월한 성적입니다."를, B이면 "우수한 성적입니다."를, C이면 "보통입니다."를, 그 외에는
"노력이 필요합니다."를 출력하라.
```

이를 if-elif-else로 작성하면 다음과 같다.

```
>>> grade = 'B'
>>> if grade == 'A':
...     print("탁월한 성적입니다.")
... elif grade == 'B':
...     print("우수한 성적입니다.")
... elif grade == 'C':
...     print("보통입니다.")
... else:
...     print("노력이 필요합니다.")
...
우수한 성적입니다.
```

이 코드는 동작에 문제는 없지만, `grade ==` 이 반복되어 읽기에 깔끔하지 않다. 이런 경우에 match-case 문을 사용하면 훨씬 간결하게 작성할 수 있다.

```
>>> grade = 'B'
>>> match grade:
...     case 'A':
...         print("탁월한 성적입니다.")
...     case 'B':
...         print("우수한 성적입니다.")
...     case 'C':
...         print("보통입니다.")
...     case _:
...         print("노력이 필요합니다.")
...
우수한 성적입니다.
```

match 뒤에 비교할 변수를 쓰고, case 뒤에 해당 변수와 비교할 값을 적는다. 위에서부터 순서대로 비교하다가 일치하는 case를 만나면 그 case에 속한 문장을 수행한다. 어떤 case에도 일치하지 않으면 `case _`에 속한 문장을 수행한다. 여기서 `_`는 '그 외 모든 값'을 의미하는 와일드카드 패턴이다. if-else에서 else와 비슷한 역할을 한다고 보면 된다.

match-case 문의 기본 구조는 다음과 같다.

```
match 변수:
    case 패턴1:
        수행할_문장1
        ...
    case 패턴2:
        수행할_문장2
        ...
    case _:
        수행할_문장3
        ...
```

> *`case _` 는 생략할 수 있다. 생략하면 어떤 case에도 일치하지 않을 때 아무 일도 하지 않는다.*

점프 투 파이썬

여러 패턴을 하나의 case에서 처리하기

하나의 case에 여러 값을 지정할 수도 있다. `|` 기호를 사용하면 된다.

```
>>> grade = "B"
>>> match grade:
...     case "A" | "B" | "C":
...         print("합격입니다.")
...     case _:
...         print("불합격입니다.")
...
합격입니다.
```

`"A" | "B" | "C"` 는 grade의 값이 이 셋 중 하나이면 해당 case를 수행하라는 뜻이다.

비교 연산자의 연쇄 사용

파이썬에서는 수학에서처럼 비교 연산자를 연쇄적으로 사용할 수 있다.

```
>>> x = 5
>>> 1 < x < 10    # x가 1보다 크고 10보다 작은가?
True
>>> 10 ≤ x ≤ 20  # x가 10 이상 20 이하인가?
False
```

이는 다음과 같이 쓰는 것과 같다.

```
>>> x = 5
>>> (1 < x) and (x < 10)
True
>>> (10 ≤ x) and (x ≤ 20)
False
```

연쇄 비교 연산자를 사용하면 더 간결하고 읽기 쉬운 코드를 작성할 수 있다.

조건부 표현식

조건에 따라 변수에 서로 다른 값을 대입하고 싶을 때가 있다. 예를 들어 점수가 60점 이상이면 "합격", 미만이면 "불합격"이라는 메시지를 저장하고 싶다고 해보자.

```
>>> score = 85
>>> if score ⩾ 60:
...     result = "합격"
... else:
...     result = "불합격"
...
>>> print(result)
합격
```

이런 간단한 조건 분기는 파이썬의 **조건부 표현식**을 사용하면 한 줄로 표현할 수 있다.

```
>>> score = 85
>>> result = "합격" if score ⩾ 60 else "불합격"
>>> print(result)
합격
```

조건부 표현식의 기본 형태는 다음과 같다.

```
변수 = 참일_때_값 if 조건 else 거짓일_때_값
```

"조건이 참이면 '참일_때_값'을, 거짓이면 '거짓일_때_값'을 변수에 대입한다"는 의미이다.

몇 가지 예제를 더 살펴보자.

```
>>> age = 19
>>> status = "성인" if age ⩾ 18 else "미성년"
>>> print(status)
성인

>>> temperature = 25
>>> weather = "따뜻함" if temperature > 20 else "추움"
>>> print(weather)
따뜻함

>>> money = 1500
>>> transportation = "버스" if money ⩾ 1000 else "도보"
>>> print(transportation)
버스
```

조건부 표현식은 코드를 간결하게 만들어 주지만, 복잡한 조건이나 긴 표현식에는 사용하지 않는 것이 좋다. 가독성이 떨어질 수 있기 때문이다.

03-2 while문

문장을 반복해서 수행해야 할 경우 while 문을 사용한다. 그래서 while 문을 '반복문'이라고도 부른다.

- while 문의 기본 구조
- while 문 만들기
- while 문 강제로 빠져나가기
- while 문의 맨 처음으로 돌아가기
- 무한 루프
- while-else 문
- 중첩된 while 문

while 문의 기본 구조

다음은 while 문의 기본 구조이다.

```
while 조건문:
    수행할_문장1
    수행할_문장2
    수행할_문장3
    ...
```

while 문은 조건문이 참인 동안 while 문에 속한 문장들을 반복해서 수행한다.

'열 번 찍어 안 넘어가는 나무 없다'라는 속담을 파이썬 프로그램으로 만들면 다음과 같다.

```
>>> treeHit = 0
>>> while treeHit < 10:
...     treeHit = treeHit +1
...     print(f"나무를 {treeHit}번 찍었습니다.")
...     if treeHit == 10:
...         print("나무 넘어갑니다.")
...
나무를 1번 찍었습니다.
나무를 2번 찍었습니다.
나무를 3번 찍었습니다.
나무를 4번 찍었습니다.
나무를 5번 찍었습니다.
나무를 6번 찍었습니다.
나무를 7번 찍었습니다.
나무를 8번 찍었습니다.
나무를 9번 찍었습니다.
나무를 10번 찍었습니다.
나무 넘어갑니다.
```

while 문의 조건문은 `treeHit < 10` 이다. 즉, treeHit가 10보다 작은 동안 while 문에 포함된 문장들을 계속 수행한다. while 문 안의 문장을 보면 `treeHit = treeHit + 1` 로 treeHit 값이 1씩 증가한다. 그리고 나무를 treeHit번만큼 찍었다는 문장을 출력하고, treeHit가 10이 되면 "나무 넘어갑니다."를 출력한다. 그러고 나면 `treeHit < 10` 조건문이 거짓이 되므로 while 문을 빠져나간다.

> *`treeHit = treeHit + 1` 은 프로그래밍에서 매우 자주 사용하는 기법이다. treeHit 값을 1씩 증가시킬 목적으로 사용하며, `treeHit += 1` 처럼 작성해도 된다.*

다음은 while 문이 반복되는 과정을 순서대로 정리한 표이다. 이 긴 과정을 소스 코드 단 5줄로 만들 수 있다니 놀랍지 않은가?

treeHit	조건문	조건 판단	수행하는 문장	while문
0	0 < 10	참	나무를 1번 찍었습니다.	반복
1	1 < 10	참	나무를 2번 찍었습니다.	반복
2	2 < 10	참	나무를 3번 찍었습니다.	반복
3	3 < 10	참	나무를 4번 찍었습니다.	반복
4	4 < 10	참	나무를 5번 찍었습니다.	반복
5	5 < 10	참	나무를 6번 찍었습니다.	반복
6	6 < 10	참	나무를 7번 찍었습니다.	반복
7	7 < 10	참	나무를 8번 찍었습니다.	반복
8	8 < 10	참	나무를 9번 찍었습니다.	반복
9	9 < 10	참	나무를 10번 찍었습니다. 나무 넘어갑니다.	반복
10	10 < 10	거짓		종료

while 문 만들기

이번에는 여러 가지 선택지 중 하나를 선택해서 입력받는 예제를 만들어 보자. 먼저 다음과 같이 여러 줄짜리 문자열을 만든다.

```
>>> prompt = """
... 1. Add
... 2. Del
... 3. List
... 4. Quit
...
... Enter number: """
>>>
```

이어서 number 변수에 0을 먼저 대입한다. 이렇게 변수를 먼저 설정해 놓지 않으면 while 문의 조건문인 `number ≠ 4`에서 변수가 존재하지 않는다는 오류가 발생한다.

```
>>> number = 0
>>> while number ≠ 4:
...     print(prompt)
...     number = int(input())
...

1. Add
2. Del
3. List
4. Quit

Enter number:
```

while 문을 보면 number가 4가 아닌 동안 prompt를 출력하고 사용자로부터 번호를 입력받는다. 사용자가 4를 입력하지 않으면 계속해서 prompt를 출력한다.

> *`number = int(input())` 는 사용자의 숫자 입력을 받아들이는 것이라고만 알아 두자. int나 input 함수에 대한 내용은 내장 함수 부분에서 자세히 다룬다.*

```
Enter number:
1

1. Add
2. Del
3. List
4. Quit
```

4를 입력하면 조건문이 거짓이 되어 while 문을 빠져나간다.

```
Enter number:
4
>>>
```

while 문 강제로 빠져나가기

while 문은 조건문이 참인 동안 while 문 안의 내용을 반복적으로 수행한다. 하지만 강제로 while 문을 빠져나가고 싶을 때가 있다.

커피 자판기를 예로 들어 보자. 자판기 안에 커피가 충분히 있을 때 동전을 넣으면 커피가 나온다. 그런데 자판기가 제대로 작동하려면 커피가 얼마나 남았는지 항상 검사해야 한다. 만약 커피가 떨어졌다면 판매를 중단하고 '판매 중지' 문구를 보여 주어야 한다. 이렇게 판매를 강제로 멈추게 하는 것이 바로 break 문이다.



다음은 커피 자판기 이야기를 파이썬 프로그램으로 표현한 것이다.

```
>>> coffee = 10
>>> money = 300
>>> while money:
...     print("돈을 받았으니 커피를 줍니다.")
...     coffee = coffee -1
...     print(f"남은 커피의 양은 {coffee}개입니다.")
...     if coffee == 0:
...         print("커피가 다 떨어졌습니다. 판매를 중지합니다.")
...         break
...
```

money가 300으로 고정되어 있고 `while money:`에서 조건문인 money는 0이 아니기 때문에 항상 참이다. 따라서 무한히 반복되는 무한 루프를 돌게 된다. while 문의 내용을 한 번 수행할 때마다 `coffee = coffee - 1`에 의해 coffee의 개수가 1개씩 줄어든다. coffee가 0이 되면 `if coffee == 0:` 문장에서 `coffee == 0`이 참이 되므로 "커피가 다 떨어졌습니다. 판매를 중지합니다."가 출력되고 break 문이 호출되어 while 문을 빠져나간다.

하지만 실제 자판기는 위 예처럼 작동하지는 않을 것이다. 다음은 자판기의 실제 작동 과정과 비슷하게 만든 예이다. 조금 복잡하므로 대화형 인터프리터를 사용하지 말고 IDLE 에디터를 사용해서 작성해 보자.

```
# coffee.py
coffee = 10
while True:
    money = int(input("돈을 넣어 주세요: "))
    if money == 300:
        print("커피를 줍니다.")
        coffee = coffee -1
    elif money > 300:
        print(f"거스름돈 {money - 300}를 주고 커피를 줍니다.")
        coffee = coffee -1
    else:
        print("돈을 다시 돌려주고 커피를 주지 않습니다.")
        print(f"남은 커피의 양은 {coffee}개입니다.")
    if coffee == 0:
        print("커피가 다 떨어졌습니다. 판매를 중지 합니다.")
        break
```

위 프로그램의 흐름을 간단히 살펴보자. `while True:`로 무한 루프를 돌면서 사용자에게 돈을 입력받는다. 입력한 금액이 300원이면 커피를 주고, 300원보다 많으면 거스름돈을 돌려주고 커피를 준다. 300원보다 적으면 돈을 돌려준다. 커피를 줄 때마다 `coffee` 값이 1씩 줄어들고, 0이 되면 break로 반복을 종료한다. `money = int(input("돈을 넣어 주세요: "))` 문장이 이해되지 않는다면, 사용자로부터 값을 입력받아 money 변수에 대입하는 것이라고만 알아 두자.

coffee.py 파일을 저장한 후 명령 프롬프트 창을 열어 프로그램을 직접 실행해 보자.

> *프로그램 소스를 에디터로 작성해서 실행하는 방법이 기억나지 않는다면 01-6절을 참고하기 바란다.*

다음과 같은 입력란이 나타난다.

```
C:\doit>python coffee.py
돈을 넣어 주세요:
```

입력란에 여러 숫자를 입력해 보면서 결과를 확인하자.

```
돈을 넣어 주세요: 500
거스름돈 200를 주고 커피를 줍니다.
돈을 넣어 주세요: 300
커피를 줍니다.
돈을 넣어 주세요: 100
돈을 다시 돌려주고 커피를 주지 않습니다.
남은 커피의 양은 8개입니다.
돈을 넣어 주세요:
```

while 문의 맨 처음으로 돌아가기

while 문 안의 문장을 수행할 때 입력 조건을 검사해서 조건에 맞지 않으면 while 문을 빠져나간다. 그런데 프로그래밍을 하다 보면 while 문을 빠져나가지 않고 while 문의 맨 처음(조건문)으로 다시 돌아가게 만들고 싶은 경우가 있다. 이때 사용하는 것이 바로 continue 문이다.

1부터 10까지의 숫자 중에서 홀수만 출력하는 것을 while 문을 사용해서 작성해 보자.

```
>>> a = 0
>>> while a < 10:
...     a = a + 1
...     if a % 2 == 0: continue
...     print(a)
...
1
3
5
7
9
```

a가 10보다 작은 동안 a는 1씩 계속 증가한다. `a % 2 == 0` (a를 2로 나누었을 때 나머지가 0인 경우)이 참이 되는 경우는 a가 짝수일 때이다. 즉, a가 짝수이면 continue 문을 수행한다. continue 문은 while 문의 맨 처음인 조건문(a < 10)으로 돌아가게 하는 명령어이다. 따라서 a가 짝수이면 print(a) 문장은 수행되지 않는다.

무한 루프

무한 루프(endless loop)는 무한히 반복한다는 의미이다. 서버, 이벤트 처리, 입력 대기 같은 상황에서 무한 루프는 자주 사용된다.

파이썬에서 무한 루프는 while 문으로 구현할 수 있다. 다음은 무한 루프의 기본 형태이다.

```
while True:
    수행할_문장1
    수행할_문장2
    ...
```

while 문의 조건문이 True이므로 항상 참이 된다. 따라서 while 문 안에 있는 문장들은 무한히 수행된다.

다음은 무한 루프의 예이다.

```
>>> while True:
...     print("Ctrl+C를 눌러야 while문을 빠져나갈 수 있습니다.")
...
Ctrl+C를 눌러야 while문을 빠져나갈 수 있습니다.
Ctrl+C를 눌러야 while문을 빠져나갈 수 있습니다.
Ctrl+C를 눌러야 while문을 빠져나갈 수 있습니다.
....
```

위 문장은 영원히 출력된다. 하지만 이 예처럼 아무 의미 없이 무한 루프를 돌리는 경우는 거의 없다. `[Ctrl+C]`를 눌러 빠져나가자.

while-else 문

while 문 뒤에 else를 붙일 수 있다. while 문이 조건이 거짓이 되어 정상적으로 끝나면 else 절이 실행되고, break로 빠져나가면 else 절은 실행되지 않는다.

```
>>> count = 0
>>> while count < 3:
...     print(f"카운트: {count}")
...     count += 1
... else:
...     print("while 문이 정상 종료되었습니다.")
...
카운트: 0
카운트: 1
카운트: 2
while 문이 정상 종료되었습니다.
```

만약 위 예제에서 break로 빠져나가면 else 절의 문장은 출력되지 않는다.

```
>>> count = 0
>>> while count < 5:
...     if count == 2:
...         break
...     print(f"카운트: {count}")
...     count += 1
... else:
...     print("while 문이 정상 종료되었습니다.")
...
카운트: 0
카운트: 1
```

> *while-else 문은 자주 사용하지는 않지만, 반복이 끝까지 완료되었는지를 확인할 때 유용하다.*

중첩된 while 문

while 문 안에 또 다른 while 문을 사용할 수 있다. 이를 중첩된 while 문이라고 한다. 다음은 중첩된 while 문으로 구구단 2단과 3단을 출력하는 예이다.

```
>>> i = 2
>>> while i <= 3:
...     j = 1
...     while j <= 9:
...         print(f"{i} x {j} = {i*j}")
...         j += 1
...     i += 1
...
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
...
3 x 8 = 24
3 x 9 = 27
```

바깥쪽 while 문은 단(i)을 반복하고, 안쪽 while 문은 각 단의 곱셈(j)을 반복한다. 이처럼 중첩된 while 문은 바깥쪽 반복 한 번마다 안쪽 반복을 처음부터 끝까지 수행한다.

> *중첩된 while 문에서 break나 continue를 사용하면 가장 가까운 while 문에만 영향을 준다.*

03-3 for문

파이썬의 직관적인 특징을 가장 잘 보여 주는 것이 바로 for 문이다. while 문과 비슷한 반복문인 for 문은 문장 구조가 한눈에 들어온다는 장점이 있다. for 문을 잘 사용하면 프로그래밍이 즐거워질 것이다.

- for 문의 기본 구조
- 예제를 통해 for 문 이해하기
    - 1. 전형적인 for 문
    - 2. 다양한 for 문의 사용
    - 3. for 문의 응용
- for 문과 continue 문
- for 문과 함께 자주 사용하는 range 함수
    - range 함수의 예시 살펴보기
    - for와 range를 이용한 구구단
- 리스트 컴프리헨션 사용하기
- for 문과 break 문
- for-else 문
- enumerate 함수 활용하기
- zip 함수로 여러 리스트 함께 순회하기

for 문의 기본 구조

for 문의 기본 구조는 다음과 같다.

```
for 변수 in 리스트(또는 튜플, 문자열):
    수행할_문장1
    수행할_문장2
    ...
```

리스트나 튜플, 문자열의 첫 번째 요소부터 마지막 요소까지 차례로 변수에 대입되어 '수행할_문장1', '수행할_문장2' 등이 수행된다.

예제를 통해 for 문 이해하기

for 문은 예제를 통해 살펴보는 것이 가장 알기 쉽다. 다음 예제를 직접 입력해 보자.

1. 전형적인 for 문

```
>>> test_list = ['one', 'two', 'three']
>>> for i in test_list:
...     print(i)
...
one
two
three
```

`['one', 'two', 'three']` 리스트의 첫 번째 요소인 `'one'` 이 먼저 i 변수에 대입된 후 `print(i)` 문장을 수행한다. 다음에 두 번째 요소 `'two'`가 i 변수에 대입된 후 `print(i)` 문장을 수행하고, 리스트의 마지막 요소까지 이것을 반복한다.

2. 다양한 for 문의 사용

```
>>> a = [(1,2), (3,4), (5,6)]
>>> for (first, last) in a:
...     print(first + last)
...
3
7
11
```

위 예는 a 리스트의 요솟값이 튜플이기 때문에 각각의 요소가 자동으로 `(first, last)` 변수에 대입된다.

3. for 문의 응용

for 문의 쓰임새를 알기 위해 다음 문제를 생각해 보자.

```
총 5명의 학생이 시험을 보았는데 시험 점수가 60점 이상이면 합격이고 그렇지 않으면 불합격이다. 합격인지, 불합격인지 결과를 보여 주시오.
```

먼저 학생 5명의 시험 점수를 리스트로 표현해 보자.

```
marks = [90, 25, 67, 45, 80]
```

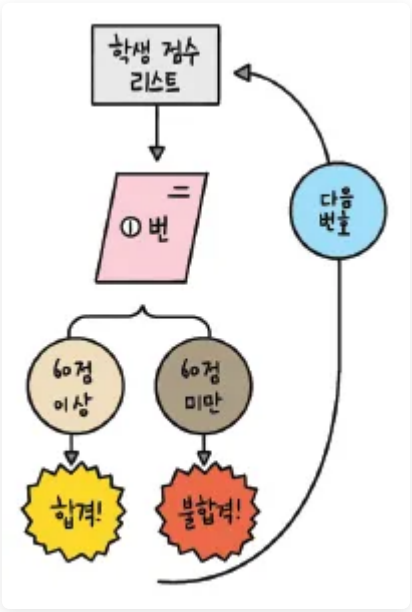
1번 학생은 90점이고 5번 학생은 80점이다.

이 점수를 차례로 검사해서 합격했는지, 불합격했는지 통보해 주는 프로그램을 만들어 보자. IDLE 에디터로 작성하자.

```
# marks1.py
marks = [90, 25, 67, 45, 80]   # 학생들의 시험 점수 리스트

number = 0   # 학생에게 붙여 줄 번호
for mark in marks:   # 90, 25, 67, 45, 80을 순서대로 mark에 대입
    number = number + 1
    if mark >= 60:
        print(f"{number}번 학생은 합격입니다.")
    else:
        print(f"{number}번 학생은 불합격입니다.")
```

각각의 학생에게 번호를 붙여 주기 위해 number 변수를 사용했다. 점수 리스트 marks에서 차례로 점수를 꺼내어 mark라는 변수에 대입하고 for 문 안의 문장들을 수행한다. for 문이 한 번씩 수행될 때마다 number는 1씩 증가한다.



이 프로그램을 실행하면 mark가 60 이상일 때 합격 메시지를 출력하고 60을 넘지 않을 때 불합격 메시지를 출력한다. 명령 프롬프트 창을 열어 실행해 보자.

```
C:\doit>python marks1.py
1번 학생은 합격입니다.
2번 학생은 불합격입니다.
3번 학생은 합격입니다.
4번 학생은 불합격입니다.
5번 학생은 합격입니다.
```

for 문과 continue 문

while 문에서 살펴본 continue 문을 for 문에서도 사용할 수 있다. for 문 안의 문장을 수행하는 도중 continue 문을 만나면 for 문의 처음으로 돌아간다.

앞에서 만든 for 문 응용 예제를 사용해서 60점 이상인 사람에게는 축하 메시지를 보내고 나머지 사람에게는 아무런 메시지도 전하지 않는 프로그램을 IDLE 에디터로 작성해 보자.

```
# marks2.py
marks = [90, 25, 67, 45, 80]

number = 0
for mark in marks:
    number = number + 1
    if mark < 60:
        continue
    print(f"{number}번 학생 축하합니다. 합격입니다.")
```

점수가 60점 미만인 학생인 경우에는 `mark < 60` 이 참이 되어 continue 문이 수행된다. 따라서 축하 메시지를 출력하는 print 문을 수행하지 않고 for 문의 처음으로 돌아간다.

```
C:\doit>python marks2.py
1번 학생 축하합니다. 합격입니다.
3번 학생 축하합니다. 합격입니다.
5번 학생 축하합니다. 합격입니다.
```

for 문과 함께 자주 사용하는 range 함수

for 문은 숫자 리스트를 자동으로 만들어 주는 range 함수와 함께 사용하는 경우가 많다. 다음은 range 함수의 간단한 사용법이다.

```
>>> a = range(10)
>>> a
range(0, 10)
```

`range(10)`은 0부터 10 미만의 숫자를 포함하는 range 객체를 만들어 준다.

시작 숫자와 끝 숫자를 지정하려면 `range(시작_숫자, 끝_숫자)` 형태를 사용하는데, 이때 끝 숫자는 포함되지 않는다.

```
>>> a = range(1, 11)
>>> a
range(1, 11)
```

range 함수의 예시 살펴보기

for와 range 함수를 사용하면 1부터 10까지 더하는 것을 다음과 같이 쉽게 구현할 수 있다.

```
>>> add = 0
>>> for i in range(1, 11):
...     add = add + i
...
>>> print(add)
55
```

`range(1, 11)`은 숫자 1부터 10까지(1 이상 11 미만)의 숫자를 데이터로 가지는 객체이다. 따라서 위 예에서 i 변수에 숫자가 1부터 10까지 하나씩 차례로 대입되면서 `add = add + i` 문장을 반복적으로 수행하고 add는 최종적으로 55가 된다.

또한 앞에서 살펴본 합격 축하 문장을 출력하는 예제도 range 함수를 사용해서 다음과 같이 바꿀 수 있다.

```
# marks3.py
marks = [90, 25, 67, 45, 80]
for number in range(len(marks)):
    if marks[number] < 60:
        continue
    print(f"{number+1}번 학생 축하합니다. 합격입니다.")
```

len 함수는 리스트 안의 요소 개수를 반환하는 함수이다. 따라서 `len(marks)` 는 5, `range(len(marks))` 는 `range(5)` 가 된다. number 변수에는 차례로 0부터 4까지의 숫자가 대입되고 `marks[number]` 는 차례대로 90, 25, 67, 45, 80 값을 가지게 된다. 결과는 marks2.py 예제와 동일하다.

for와 range를 이용한 구구단

for와 range 함수를 사용하면 소스 코드 단 4줄만으로 구구단을 출력할 수 있다. 들여쓰기에 주의하면서 입력해 보자.

```
>>> for i in range(2,10):        # 1번 for문
...     for j in range(1, 10):   # 2번 for문
...         print(i*j, end=" ")
...     print('')
...
2 4 6 8 10 12 14 16 18
3 6 9 12 15 18 21 24 27
4 8 12 16 20 24 28 32 36
5 10 15 20 25 30 35 40 45
6 12 18 24 30 36 42 48 54
7 14 21 28 35 42 49 56 63
8 16 24 32 40 48 56 64 72
9 18 27 36 45 54 63 72 81
```

위 예를 보면 for 문을 두 번 사용했다. 1번 for 문에서 2부터 9까지의 숫자(`range(2, 10)`)가 차례대로 i에 대입된다. i가 처음 2일 때 2번 for 문을 만나게 된다. 2번 for 문에서 1부터 9까지의 숫자(`range(1, 10)`)가 j에 대입되고 그다음 문장인 `print(i*j, end=" ")` 를 수행한다. 따라서 i가 2일 때 `2 * 1, 2 * 2, 2 * 3, … 2 * 9` 까지 차례대로 수행되며 그 값을 출력하게 된다. i가 3일 때 역시 2일 때와 마찬가지로 수행되고, i가 9일 때까지 계속 반복된다.



`print(i*j, end=" ")` 와 같이 print 함수에 end 파라미터를 설정한 이유는 해당 결괏값을 출력할 때 다음 줄로 넘기지 않고 그 줄에 계속 출력하기 위해서이다. 그다음에 이어지는 `print('')` 는 2단, 3단 등을 구분하기 위해 사용했다. 두 번째 for 문이 끝나면 결괏값을 다음 줄부터 출력하게 하는 역할을 한다.

> - *print 문의 end 매개변수에는 줄바꿈 문자(`\n`)가 기본값으로 설정되어 있다.*
> - *print 문은 04-2에서 자세히 다룬다.*

리스트 컴프리헨션 사용하기

리스트 안에 for 문을 포함하는 리스트 컴프리헨션(list comprehension)을 사용하면 좀 더 편리하고 직관적인 프로그램을 만들 수 있다. 다음 예제를 살펴보자.

```
>>> a = [1,2,3,4]
>>> result = []
>>> for num in a:
...     result.append(num*3)
...
>>> print(result)
[3, 6, 9, 12]
```

위 예제에서는 a 리스트의 각 항목에 3을 곱한 결과를 result 리스트에 담았다.

리스트 컴프리헨션을 사용하면 다음과 같이 간단하게 작성할 수 있다.

```
>>> a = [1,2,3,4]
>>> result = [num * 3 for num in a]
>>> print(result)
[3, 6, 9, 12]
```

만약 [1, 2, 3, 4] 중에서 짝수에만 3을 곱하여 담고 싶다면 리스트 컴프리헨션 안에 if 조건문을 사용하면 된다.

```
>>> a = [1,2,3,4]
>>> result = [num * 3 for num in a if num % 2 == 0]
>>> print(result)
[6, 12]
```

리스트 컴프리헨션의 문법은 다음과 같다. if 조건문 부분은 앞의 예제에서 볼 수 있듯이 생략할 수 있다.

```
[표현식 for 항목 in 반복_가능_객체 if 조건문]
```

조금 복잡하지만, for 문을 2개 이상 사용하는 것도 가능하다. for 문을 여러 개 사용할 때의 문법은 다음과 같다.

```
[표현식 for 항목1 in 반복_가능_객체1 if 조건문1
      for 항목2 in 반복_가능_객체2 if 조건문2
      ...
      for 항목n in 반복_가능_객체n if 조건문n]
```

만약 구구단의 모든 결과를 리스트에 담고 싶다면 리스트 컴프리헨션을 사용하여 다음과 같이 간단하게 구현할 수 있다.

```
>>> result = [x*y for x in range(2,10)
...               for y in range(1,10)]
>>> print(result)
[2, 4, 6, 8, 10, 12, 14, 16, 18, 3, 6, 9, 12, 15, 18, 21, 24, 27, 4, 8, 12, 16,
20, 24, 28, 32, 36, 5, 10, 15, 20, 25, 30, 35, 40, 45, 6, 12, 18, 24, 30, 36, 42
, 48, 54, 7, 14, 21, 28, 35, 42, 49, 56, 63, 8, 16, 24, 32, 40, 48, 56, 64, 72,
9, 18, 27, 36, 45, 54, 63, 72, 81]
```

for 문과 break 문

while 문에서와 마찬가지로 for 문에서도 break 문을 사용할 수 있다. break 문은 for 문을 강제로 빠져나가고 싶을 때 사용한다.

```
>>> for i in range(10):
...     if i == 5:
...         break
...     print(i)
...
0
1
2
3
4
```

위 예제에서 i가 5가 되면 break 문이 실행되어 for 문을 빠져나간다.

for-else 문

for 문 뒤에 else를 붙일 수 있다. for 문이 끝까지 수행되면 else 절이 실행되고, break로 빠져나가면 else 절은 실행되지 않는다.

```
>>> for i in range(5):
...     print(i)
... else:
...     print("for 문이 정상 종료되었습니다.")
...
0
1
2
3
4
for 문이 정상 종료되었습니다.
```

만약 break로 빠져나가면 else 절의 문장은 출력되지 않는다.

```
>>> for i in range(5):
...     if i == 3:
...         break
...     print(i)
... else:
...     print("for 문이 정상 종료되었습니다.")
...
0
1
2
```

> *for-else 문은 자주 사용하지는 않지만, 리스트에서 원하는 값을 찾았는지 여부를 판단할 때 유용하다. break 없이 끝났다면 "찾지 못했다"는 의미로 else 절을 활용할 수 있다.*

enumerate 함수 활용하기

리스트의 순서(인덱스)와 값을 함께 구하고 싶을 때는 enumerate 함수를 사용하면 편리하다.

```
>>> fruits = ['apple', 'banana', 'orange']
>>> for i, fruit in enumerate(fruits):
...     print(f"{i}: {fruit}")
...
0: apple
1: banana
2: orange
```

enumerate는 0부터 시작하는 인덱스 번호를 자동으로 생성해 준다. 시작 번호를 변경하고 싶다면 다음과 같이 사용할 수 있다.

```
>>> fruits = ['apple', 'banana', 'orange']
>>> for i, fruit in enumerate(fruits, 1):  # 1부터 시작
...     print(f"{i}: {fruit}")
...
1: apple
2: banana
3: orange
```

zip 함수로 여러 리스트 함께 순회하기

두 개 이상의 리스트를 동시에 순회하고 싶을 때는 zip 함수를 사용한다.

```
>>> names = ['홍길동', '김철수', '이영희']
>>> scores = [85, 92, 78]
>>> for name, score in zip(names, scores):
...     print(f"{name}: {score}점")
...
홍길동: 85점
김철수: 92점
이영희: 78점
```

세 개 이상의 리스트도 함께 순회할 수 있다.

```
>>> names = ['홍길동', '김철수', '이영희']
>>> korean = [85, 92, 78]
>>> english = [90, 88, 95]
>>> for name, kor, eng in zip(names, korean, english):
...     print(f"{name}: 국어 {kor}점, 영어 {eng}점")
...
홍길동: 국어 85점, 영어 90점
김철수: 국어 92점, 영어 88점
이영희: 국어 78점, 영어 95점
```

지금까지 프로그램 흐름을 제어하는 if 문, while 문, for 문에 대해 알아보았다. 아마도 여러분은 while 문과 for 문을 보면서 2가지가 매우 비슷하다는 느낌을 받았을 것이다. 실제로 for 문으로 작성한 코드를 while 문으로 바꿀 수 있는 경우도 많고 while 문을 for 문으로 바꾸어서 사용할 수 있는 경우도 많다.

03장 되새김 문제

죄송합니다. 되새김 문제는 위키독스 웹 사이트에서 비공개 처리되어 있습니다.
되새김 문제에 대한 상세한 내용은 유료 E-book 또는 종이책에서 확인할 수 있습니다.

- 책 구입 안내